

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases

Functions – Case Based Learning

Name: K. Dedeepya

Reg no: 192324252

1. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Introduction

Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are machine learning techniques used for function approximation. Both methods give more importance to nearby data points, but they differ in how locality is represented, how training is performed, computational cost, and prediction smoothness.

For example, consider a dataset containing the relationship between hours studied and examination marks. If the relationship is non-linear, both LWR and RBF can approximate the curve.

Example Dataset

Hours Studied	Marks
1	35
2	42
3	50
4	61
5	68

In LWR, when predicting the marks for a particular number of study hours, nearby observations receive higher weights. In an RBF network, radial basis functions are placed around selected centers, and their activation depends on the distance between the input and the center.

Comparison

Feature	Locally Weighted Regression	Radial Basis Function Network
Basic idea	Fits a local regression model around each query point	Uses radial basis functions as hidden units
Locality	Based directly on distance from query point	Based on distance from RBF centers

Training	Little or no permanent global training	Requires learning centers, widths and output weights
Prediction	Performs local regression for every new input	Passes input through RBF units and calculates weighted output
Memory	Stores training data	Stores learned parameters/centers
Smoothness	Depends strongly on bandwidth	Usually provides smooth smoothness
Adaptability	Easily adapts when new data is added	Usually requires retraining/update
Best suited for	Small datasets and locally changing relationships	Non-linear function approximation

Locality

Locally Weighted Regression: LWR assigns a higher weight to data points closer to the query point. A Gaussian weighting function can be represented as:

$$w(x_i) = \exp(-\|x - x_i\|^2 / 2\tau^2)$$

where:

- x = query point
- x_i = training data point
- τ = bandwidth parameter

A small τ gives strong locality, while a large τ considers more data points.

RBF Network: An RBF network uses functions such as:

$$\phi(x) = \exp(-\|x - c\|^2 / 2\sigma^2)$$

where:

- c = center of the RBF
- σ = width of the function

The final output is generally: $y = \sum w_i \phi_i(x) + b$

Computational Cost

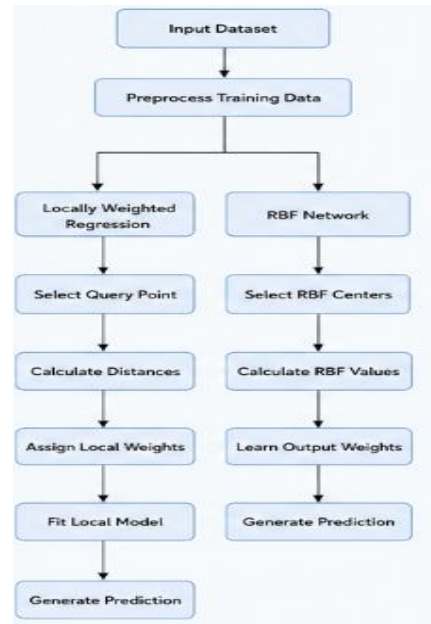
LWR does not build a fixed global model. For every new input, it calculates distances, weights, and a local regression model. Therefore, prediction can become expensive when the training dataset is large.

RBF networks perform more computation during training because the centers and weights have to be learned. However, after training, prediction is generally faster because the learned RBF parameters are reused.

Smoothness of Predictions

LWR can produce smooth predictions when an appropriate bandwidth is selected. However, a very small bandwidth may produce unstable predictions, while a very large bandwidth may over smooth the data. RBF networks normally generate smooth non-linear functions because radial basis functions overlap and contribute gradually according to distance.

Flowchart



Analysis

For a small dataset where the relationship changes significantly from one region to another, LWR is useful because it adapts directly to local observations. For larger non-linear function approximation problems, RBF networks are generally more practical because they learn a compact set of basic functions.

Conclusion

Both LWR and RBF networks use locality to approximate non-linear functions. LWR provides high adaptability and is simple for small datasets, but its prediction cost can become high. RBF networks require more training but provide efficient and smooth predictions after learning. Therefore, LWR is suitable for small, dynamically changing datasets, while RBF networks are suitable for repeated prediction and non-linear function approximation.

2. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyse assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

Introduction

Naïve Bayes and Logistic Regression are commonly used supervised learning algorithms for text classification. A typical application is classifying an email as Spam or Not Spam.

For example:

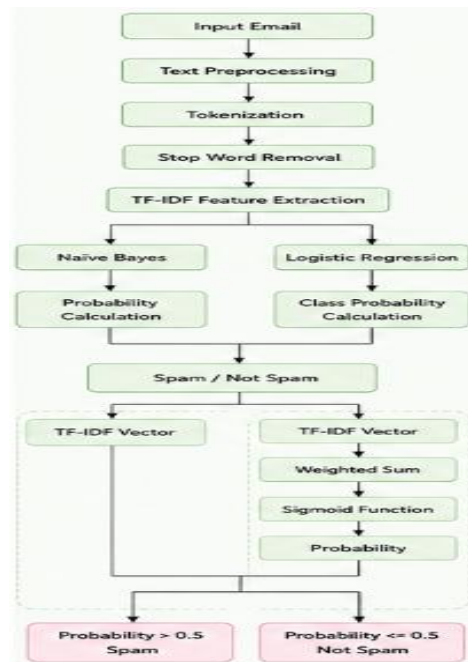
- "Congratulations! You won a free prize" → Spam
- "Please attend the project meeting tomorrow" → Not Spam

Both algorithms can work efficiently with high-dimensional text data such as TF-IDF or word-count vectors.

Text Classification Process

The text must first be converted into numerical features. One common approach is TF-IDF (Term Frequency-Inverse Document Frequency).

Example:



Comparison

Feature	Naïve Bayes	Logistic Regression
Type	Probabilistic classifier	Discriminative classifier
Main principle	Bayes theorem	Logistic/sigmoid function
Feature independence	Assumes features are conditionally independent	Does not require independence assumption
Output	Class probabilities	Class probabilities
Training speed	Very fast	Fast
High-dimensional data	Performs very well	Performs very well
Correlated features	Can reduce effectiveness	Usually handles them better
Dataset requirement	Works well with smaller datasets	Usually benefits from sufficient training data

Naïve Bayes

Naïve Bayes is based on Bayes' theorem: $P(C|X) = \frac{P(X|C) P(C)}{P(X)}$

For spam detection: **P (Spam | Words)**

is compared with: **P (Not Spam | Words)**

The class with the higher probability is selected.

The "naïve" assumption is that features are conditionally independent given the class. In text classification, this means the occurrence of one word is treated as independent of other words when the class is known.

Logistic Regression

Logistic Regression predicts the probability of a class using the sigmoid function:

$$P(y=1|x) = 1 / (1 + e^{-z})$$

where: $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$

For spam detection, the model calculates the probability that an email belongs to the spam class.

Assumption About Feature Independence

Naïve Bayes assumes that features are conditionally independent. This assumption is not completely true for natural language because words can depend on one another.

For example: "credit card" has a relationship between the two words.

Logistic Regression does not require the same independence assumption and can learn weights for individual features.

Computational Efficiency

Naïve Bayes is generally faster because it mainly calculates class probabilities from feature frequencies. Logistic Regression requires optimization of model weights, commonly using gradient-based optimization. Therefore, training can be more computationally expensive than Naïve Bayes, especially with a very large number of features.

High-Dimensional Data

Text datasets can contain thousands or millions of features. Both algorithms can handle sparse high-dimensional vectors effectively.

Naïve Bayes is particularly useful when:

- Training data is limited.
- Very fast training is required.
- The dataset contains many text features.

Logistic Regression is preferable when:

- More training data is available.
- Feature relationships are important.
- Better decision boundaries are required.

Python Program

```

101 from sklearn.feature_extraction.text import TfidfVectorizer
102 from sklearn.model_selection import train_test_split
103 from sklearn.naive_bayes import MultinomialNB
104 from sklearn.linear_model import LogisticRegression
105 from sklearn.metrics import accuracy_score
106 email = [
107     "Win a free lottery prize",
108     "Congratulations you won money",
109     "Claim your free gift now",
110     "Meeting is scheduled tomorrow",
111     "Please submit the project report",
112     "Attend the class at 10 am"
113 ]
114 labels = [1, 1, 1, 0, 0, 0]
115 vectorizer = TfidfVectorizer()
116 X = vectorizer.fit_transform(emails)
117 X_train, X_test, y_train, y_test = train_test_split(
118     X, labels, test_size=0.3, random_state=42)
119
120 nb = MultinomialNB()
121 nb.fit(X_train, y_train)
122 nb_pred = nb.predict(X_test)
123 lr = LogisticRegression()
124 lr.fit(X_train, y_train)
125 lr_pred = lr.predict(X_test)
126 print("Naive Bayes Accuracy:",
127       accuracy_score(y_test, nb_pred))
128 print("Logistic Regression Accuracy:",
129       accuracy_score(y_test, lr_pred))

```

Analysis

Naïve Bayes is usually preferred when speed and simplicity are important. Logistic Regression can be preferable when the features are correlated and a more flexible discriminative boundary is required.

Conclusion

For spam detection, both algorithms are effective. Naïve Bayes is faster and works very well with limited training data and high-dimensional text, while Logistic Regression provides a flexible decision boundary and does not rely on the feature independence assumption. Therefore, the choice depends on dataset size, computational requirements, and classification performance.

9. Compare Genetic Algorithms and Gradient Descent Optimization for solving optimization problems. Analyse differences in convergence behaviour, ability to escape local minima, computational cost, and suitability for different types of objective functions. Provide examples where each method is more effective.

Introduction

Genetic Algorithms (GA) and Gradient Descent (GD) are optimization techniques used to find suitable solutions for machine learning and other optimization problems. However, they use fundamentally different strategies.

- **Genetic Algorithm:** Population-based evolutionary optimization.
- **Gradient Descent:** Mathematical optimization based on the gradient of the objective function.

Comparison

Feature	Genetic Algorithm	Gradient Descent
Optimization type	Evolutionary	Gradient-based
Search strategy	Population-based	Iterative parameter update
Uses derivatives	No	Yes
Local minima	Better ability to escape	Can get trapped
Population	Required	Not required

Computational cost	High	Relatively low
Randomness	Uses stochastic operations	Usually deterministic/stochastic depending on variant
Best suited for	Complex/non-smooth optimization	Differentiable machine learning models

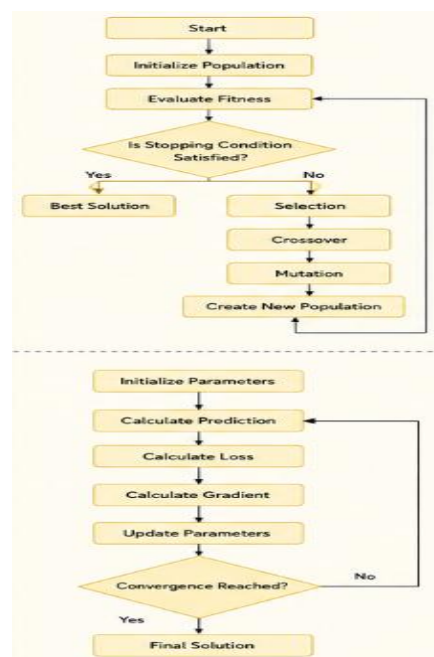
Genetic Algorithm

A Genetic Algorithm is inspired by natural evolution.

The main operations are:

1. Initialization of population
2. Fitness evaluation
3. Selection
4. Crossover
5. Mutation

Genetic Algorithm Flowchart



Gradient Descent

Gradient Descent minimizes a loss function by moving parameters in the direction opposite to the gradient.

The basic update equation is: $\theta(\text{new}) = \theta(\text{old}) - \eta \nabla J(\theta)$

where:

- θ = model parameters
- η = learning rate

- $J(\theta)$ = objective/loss function
- $\nabla J(\theta)$ = gradient

Convergence Behaviour

Gradient Descent can converge quickly when the objective function is smooth and the learning rate is properly selected.

However, convergence can be affected by:

- Learning rate
- Initial parameter values
- Shape of the objective function
- Local minima
- Saddle points

Ability to Escape Local Minima

Gradient Descent follows the gradient and can become trapped in local minima depending on the optimization problem. Genetic Algorithms maintain multiple candidate solutions simultaneously. Mutation and crossover introduce diversity, helping the algorithm explore different regions of the search space. Therefore, GA can be advantageous for highly complex search spaces.

Computational Cost

Gradient Descent is computationally efficient because it updates model parameters using gradients. Genetic Algorithms can be expensive because every generation may require evaluation of many individuals. For example, if a GA contains 100 individuals and runs for 500 generations, approximately:

$100 \times 500 = 50,000$ fitness evaluations may be required.

Examples

Genetic Algorithm

Useful for:

- Feature selection
- Scheduling
- Route optimization
- Engineering design

Example:

Find the Best Delivery Route



Generate Multiple Routes



Calculate Route Cost



Select Better Routes



Crossover + Mutation



Best Delivery Route

Gradient Descent

Useful for:

- Linear Regression
- Logistic Regression
- Neural Networks
- Deep Learning

For example, in neural network training:

Training Data



Forward Propagation



Calculate Loss



Backpropagation



Calculate Gradients



Update Weights



Repeat Until Convergence

Analysis

Genetic Algorithms are better when the objective function is non-differentiable, discontinuous, complex, or contains many local optima. Gradient Descent is more effective when the objective function is differentiable and large-scale optimization is required, such as training neural networks.

Conclusion

Genetic Algorithms provide a broad global search and do not require derivatives, but they have higher computational cost and slower convergence. Gradient Descent is computationally efficient and usually converges faster, but it depends on gradient information and may face local optimization problems. Therefore, GA is suitable for complex non-differentiable problems, whereas Gradient Descent is highly suitable for differentiable machine learning models.

12. Compare Linear Regression and Polynomial Regression for predicting housing prices. Analyse differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.

Introduction

Linear Regression and Polynomial Regression are regression techniques used to predict continuous values. In housing price prediction, the target variable is the house price, while features may include:

- Area
- Number of bedrooms
- Number of bathrooms
- Age of house
- Location score

Linear Regression assumes a linear relationship between the features and the target. Polynomial Regression extends this idea by including powers and combinations of features to model non-linear relationships.

Example Dataset

Area (sq.ft)	Price (Lakhs)
800	35
1000	42
1200	51
1400	63
1600	78

If the increase in price is approximately proportional to area, Linear Regression may work well. If the price increases at an accelerating rate, Polynomial Regression can capture that relationship.

Comparison

Feature	Linear Regression	Polynomial Regression
Relationship	Linear	Non-linear
Model complexity	Low	Higher
Formula	$y = \beta_0 + \beta_1 x$	$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots$

Interpretability	High	Lower as degree increases
Training cost	Low	Higher
Feature transformation	Not required	Polynomial features required
Large datasets	Efficient	Can become computationally expensive

Linear Regression

The simple linear regression equation is: $y = \beta_0 + \beta_1 x$

For multiple features: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$

For housing prices:



Polynomial Regression

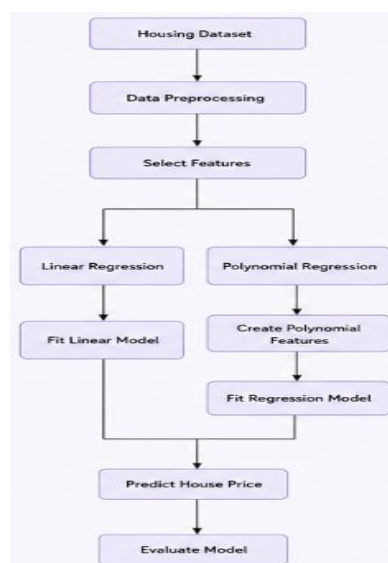
Polynomial Regression extends the feature representation:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n$$

For example, a second-degree polynomial is: $y = \beta_0 + \beta_1 x + \beta_2 x^2$

The squared term allows the model to represent a curved relationship.

Flowchart



Model Complexity

Linear Regression has relatively low complexity because it uses a linear combination of input features. Polynomial Regression increases complexity as the degree increases.

For example:

Degree 1 → Linear relationship

Degree 2 → Simple curve

Degree 3 → More complex curve

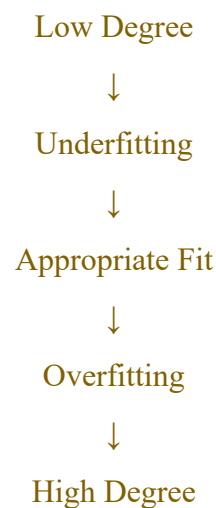
Degree 4+ → Highly flexible curve

Higher polynomial degrees can fit training data very closely but may reduce performance on unseen data.

Overfitting

Polynomial Regression has a higher risk of overfitting.

For example:



A very high-degree polynomial may learn noise instead of the actual housing-price relationship.

Linear Regression generally has lower variance and is less likely to overfit when the feature space is reasonable.

Interpretability

Linear Regression is easier to interpret.

For example: $\beta_1 = 0.05$

could indicate that a unit increase in a feature is associated with a particular change in predicted price, assuming other features remain constant. Polynomial terms are more difficult to interpret because the effect depends on the value of the feature and its higher-order terms.

Simple Python Program

```
[2]: import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X = np.array([800, 1000, 1200, 1400, 1600, 1800]).reshape(-1, 1)
y = np.array([35, 42, 51, 61, 69, 78, 95])
linear_model = LinearRegression()
linear_model.fit(X, y)
linear_prediction = linear_model.predict([[1500]])
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
poly_model = LinearRegression()
poly_model.fit(X_poly, y)
poly_prediction = poly_model.predict(
    poly.transform([[1500]])
)
print("Linear Regression Prediction:",
      linear_prediction[0])
print("Polynomial Regression Prediction:",
      poly_prediction[0])

--- Linear Regression Prediction: 72.66666666666666
Polynomial Regression Prediction: 70.133928571428551
```

Analysis

If housing prices increase almost proportionally with area, Linear Regression is preferable because it is simple, fast, and interpretable. If the relationship contains a clear curve, Polynomial Regression can provide better predictions. However, the polynomial degree must be selected carefully to avoid overfitting.

Conclusion

Linear Regression is suitable when housing price has an approximately linear relationship with the input variables. Polynomial Regression is more suitable when significant non-linear relationships exist. However, increasing polynomial degree increases complexity and overfitting risk. Therefore, the simplest model that adequately represents the data should generally be preferred.

15. Compare Naïve Bayes and Support Vector Machines (SVM) for classification problems such as sentiment analysis. Analyse differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data. Discuss scenarios where each method is preferable.

Introduction

Naïve Bayes and Support Vector Machines are effective classification algorithms for sentiment analysis. Sentiment analysis classifies text into categories such as:

- Positive
- Negative
- Neutral

Example:

"I really enjoyed this movie."



Positive

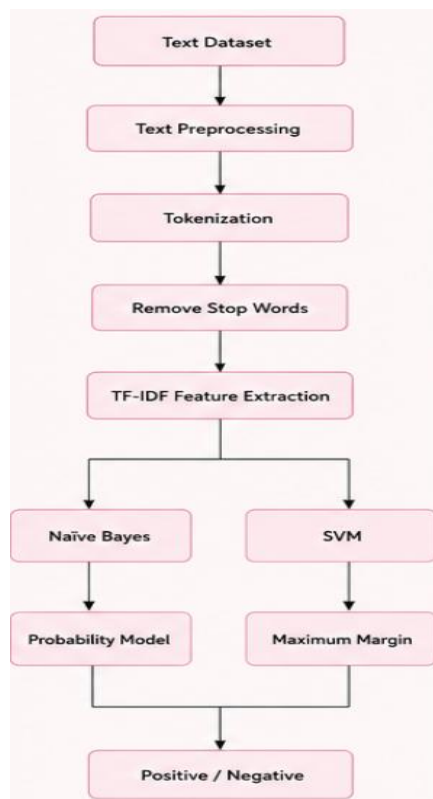
"The product quality is very poor."



Negative

Both algorithms can work effectively with high-dimensional text features such as TF-IDF vectors.

Sentiment Analysis Flowchart



Comparison

Feature	Naïve Bayes	SVM
Type	Probabilistic classifier	Maximum-margin classifier
Main principle	Bayes theorem	Finds optimal separating hyperplane
Feature independence	Assumes conditional independence	No independence assumption
Decision boundary	Probability-based	Maximum-margin boundary
Training speed	Very fast	Moderate
Prediction	Very fast	Fast after training
High-dimensional text	Excellent	Excellent
Small datasets	Performs well	Performs well

Naïve Bayes

Naïve Bayes calculates the probability that a document belongs to each class.

For sentiment analysis: $P(\text{Positive} \mid \text{Words})$ and $P(\text{Negative} \mid \text{Words})$ are calculated.

The class with the highest probability is selected.

Example:

Input:

"The movie was excellent and enjoyable"



Extract Word Features



Calculate P(Positive)

Calculate P(Negative)



$P(\text{Positive}) > P(\text{Negative})$



Positive

SVM

SVM attempts to find a decision boundary that separates classes with the maximum possible margin.

Conceptually:

Positive Documents | Negative Documents

+	+	+		-	-	-
	+	+		-	-	
		+			-	

← Maximum Margin →

Decision Boundary

The closest training points to the decision boundary are called support vectors.

Decision Boundaries

Naïve Bayes makes predictions using estimated class probabilities. SVM explicitly learns a separating hyperplane.

For a linear SVM: $\mathbf{w}^T \mathbf{x} + \mathbf{b} = 0$

where:

- $\mathbf{w}$ = weight vector
- $\mathbf{x}$ = feature vector
- $\mathbf{b}$ = bias

SVM attempts to maximize the margin between classes.

Kernel Functions

SVM can use kernel functions to transform the problem into a higher-dimensional representation.

Common kernels include:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel

The RBF kernel can be represented as: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

This allows SVM to model complex non-linear relationships.

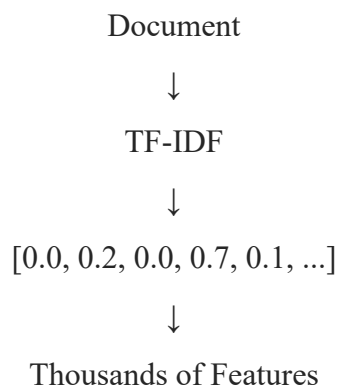
For text classification, however, a linear SVM is often highly effective because text data is usually represented using high-dimensional sparse vectors.

Computational Efficiency

Naïve Bayes is computationally efficient because training mainly involves calculating probabilities from feature counts. SVM requires solving an optimization problem to find the maximum-margin boundary. This can be more computationally demanding, especially for large datasets and non-linear kernels.

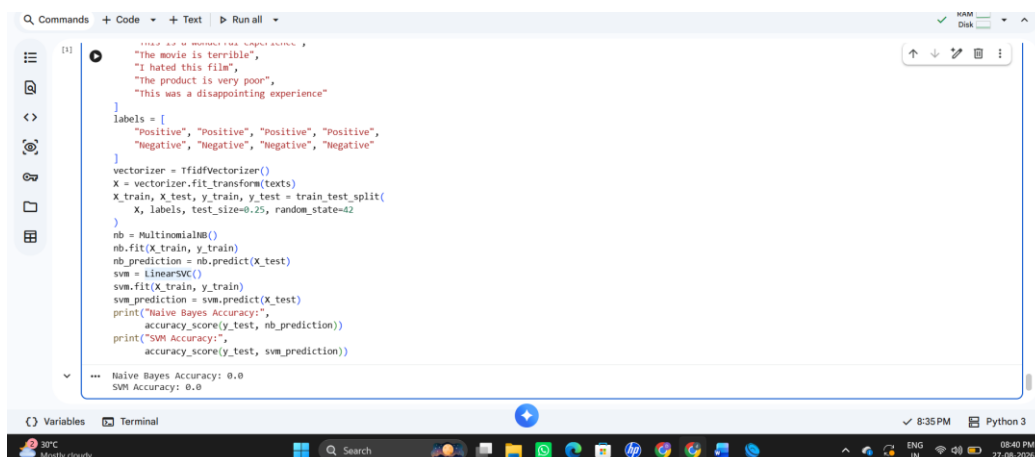
High-Dimensional Data

Text data can contain thousands of features. For example:



Both Naïve Bayes and linear SVM can handle this type of data effectively.

Simple Python Program



```
[1]: """ Naïve Bayes and SVM classification """  
texts = ["The movie is terrible",  
        "I hated this film",  
        "The product is very poor",  
        "This was a disappointing experience"]  
labels = ["Positive", "Positive", "Positive", "Positive",  
         "Negative", "Negative", "Negative", "Negative"]  
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(texts)  
X_train, X_test, y_train, y_test = train_test_split(  
    X, labels, test_size=0.25, random_state=42)  
nb = MultinomialNB()  
nb.fit(X_train, y_train)  
nb_prediction = nb.predict(X_test)  
svm = LinearSVC()  
svm.fit(X_train, y_train)  
svm_prediction = svm.predict(X_test)  
print("Naive Bayes Accuracy:",  
      accuracy_score(y_test, nb_prediction))  
print("SVM Accuracy:",  
      accuracy_score(y_test, svm_prediction))
```

Naive Bayes Accuracy: 0.0
SVM Accuracy: 0.0

Scenario-Based Analysis

Naïve Bayes is preferable when:

- Training speed is important.
- The dataset is relatively small.
- A simple probabilistic model is required.
- Very high-dimensional text features are present.

SVM is preferable when:

- High classification accuracy is required.
- The dataset has complex class boundaries.
- The feature space is high-dimensional.
- A strong discriminative classifier is required.

Conclusion

Naïve Bayes and SVM are both effective for sentiment analysis. Naïve Bayes is simple, fast, probabilistic, and highly suitable for high-dimensional text data, while SVM learns a maximum-margin decision boundary and can provide strong classification performance. For a quick and efficient sentiment classifier, Naïve Bayes is a good choice. When classification accuracy and complex decision boundaries are more important, SVM is generally preferable.